



Procesamiento de datos con Pandas

Calculando resúmenes con Pandas

Considera nuevamente la información que se ha generado con relación a la salud de las personas, específicamente para las campañas de vacunación. Dicho contenido dispone de millones de registros con información del nombre, edad, localización geográfica y padecimientos preexistentes (diabetes, obesidad, hipertensión o inmunosupresión), los cuales pueden definir la urgencia de una inoculación. Esta determinación podría quedar asentada en una nueva columna y ser parte de las entradas analizadas para el establecimiento de una estrategia nacional de vacunación en los grupos de riesgo. Pero ¿cuál sería la forma correcta de crear dicha columna para después almacenarla en un archivo?

Generar nuevas columnas en un *dataframe* es tan sencillo como asignarle valor a una variable. Los procedimientos de Pandas son tan avanzados que entienden que los cálculos se deben realizar utilizando elemento por elemento de las columnas a considerar. A continuación, abordaremos dos ejemplos para crear una columna derivada: uno utilizando solamente una columna y otro empleando dos columnas de nuestros datos originales.

Para crear una nueva columna debes utilizar una instrucción de asignación como si se tratara de un arreglo (en realidad, una serie), sin la necesidad de incorporar ciclos. Dentro de los corchetes se coloca el nombre de la nueva columna y se le asigna el valor deseado, que en muchas ocasiones depende de otra columna de nuestro *dataframe*.

Observa un ejemplo utilizando el archivo **characters.csv** (descárgalo desde el apartado “Archivos adjuntos” de la plataforma), sobre personajes de *Star Wars*, y almacenado en el *dataframe* `personajesSW`. En el siguiente *script* se crea la nueva columna `height_m` para representar la altura del personaje, en metros, y se le asignan los valores resultantes de la conversión de la columna `height`, expresada en centímetros.

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
personajesSW["height_m"] = personajesSW["height"] / 100
personajesSW.head()
```

	id	name	height	mass	hair_color	skin_color	eye_color	birth_year	gender	homeworld	species	height_m
0	1	Luke Skywalker	172	77.0	blond	fair	blue	19BBY	male	Tatooine	Human	1.72
1	2	C-3PO	167	75.0	NaN	gold	yellow	112BBY	NaN	Tatooine	Droid	1.67
2	3	R2-D2	96	32.0	NaN	white, blue	red	33BBY	NaN	Naboo	Droid	0.96
3	4	Darth Vader	202	136.0	none	white	yellow	41.9BBY	male	Tatooine	Human	2.02
4	5	Leia Organa	150	49.0	brown	light	brown	19BBY	female	Alderaan	Human	1.50

Como puedes observar, la nueva columna se ubica al final del *dataframe*, la cual contiene el resultado de dividir el valor de la columna `height` entre 100 para la totalidad de registros. Esto es ventajoso, ya que no requiere ninguna estructura de control cíclica.

En este segundo ejemplo se utilizan dos columnas del *dataframe* para obtener el índice de masa corporal de los personajes. El resultado se almacena en una nueva columna al final del *dataframe*, denominada `imc`.

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
personajesSW["imc"] = personajesSW["mass"] / (personajesSW
["height"] / 100) ** 2
personajesSW.head()
```

	id	name	height	mass	hair_color	skin_color	eye_color	birth_year	gender	homeworld	species	imc
0	1	Luke Skywalker	172	77.0	blond	fair	blue	19BBY	male	Tatooine	Human	26.027582
1	2	C-3PO	167	75.0	NaN	gold	yellow	112BBY	NaN	Tatooine	Droid	26.892323
2	3	R2-D2	96	32.0	NaN	white, blue	red	33BBY	NaN	Naboo	Droid	34.722222
3	4	Darth Vader	202	136.0	none	white	yellow	41.9BBY	male	Tatooine	Human	33.330066
4	5	Leia Organa	150	49.0	brown	light	brown	19BBY	female	Alderaan	Human	21.777778

Otra práctica de utilidad al crear columnas derivadas es el empleo de **condicionales** para el registro de valores en dependencia de su veracidad o falsedad. Para que puedas realizar este proceso, debes utilizar una notación especial, la cual puede utilizarse de la siguiente manera:

```
[valor_verdadero if condición else valor_falso for valor in columna]
```

Lo anterior se puede interpretar de la siguiente manera: colocar el **valor_verdadero** si la condición se cumple; en caso contrario, colocar el **valor_falso**. Esto para cada valor de la columna del *dataframe*. Los paréntesis cuadrados son obligatorios. Por ejemplo, puedes crear una columna derivada llamada `vaccine`, donde colocarás "Apply" si el género es igual a `female`; en caso contrario, colocarás "No" para todos los valores de género de la columna `gender` del *dataframe* `personajesSW`. De esta forma tendrías:

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
personajesSW['vaccine'] = ["Apply" if gender == "female" else
"No" for gender in personajesSW['gender']]
personajesSW.head()
```

	id	name	height	mass	hair_color	skin_color	eye_color	birth_year	gender	homeworld	species	vaccine
0	1	Luke Skywalker	172	77.0	blond	fair	blue	19BBY	male	Tatooine	Human	No
1	2	C-3PO	167	75.0	NaN	gold	yellow	112BBY	NaN	Tatooine	Droid	No
2	3	R2-D2	96	32.0	NaN	white, blue	red	33BBY	NaN	Naboo	Droid	No
3	4	Darth Vader	202	136.0	none	white	yellow	41.9BBY	male	Tatooine	Human	No
4	5	Leia Organa	150	49.0	brown	light	brown	19BBY	female	Alderaan	Human	Apply

Como puedes observar, al final del *dataframe* se creó la columna derivada llamada *vaccine*, con los valores definidos *No* y *Apply*.

También puedes encontrar situaciones donde la columna derivada involucra no únicamente los datos de cada registro, sino que además necesites un **cálculo sobre toda la columna** donde están dichos datos. Por ejemplo, considera un *dataframe* con las ventas de diferentes sucursales por estado, como se muestra a continuación:

```
import pandas as pd
ventas = pd.DataFrame(
{"Estado": ["Monterrey", "Ciudad de México", "Hidalgo",
"Guerrero"],
"Ventas": [1500, 2000, 1200, 800]
})
ventas
```

	Estado	Ventas
0	Monterrey	1500
1	Ciudad de México	2000
2	Hidalgo	1200
3	Guerrero	800

Para obtener el porcentaje que representan las ventas de cada estado con respecto al total de la empresa, podrías realizar lo siguiente:

```
ventas['porcentaje'] = ventas['Ventas'] *100 / ventas['Ventas'].sum()
ventas
```

	Estado	Ventas	porcentaje
0	Monterrey	1500	27.272727
1	Ciudad de México	2000	36.363636
2	Hidalgo	1200	21.818182
3	Guerrero	800	14.545455

Las funciones `lambda` también son empleadas en la generación de columnas derivadas. Para ejemplificar su uso, tomemos como base el siguiente *dataframe*, denominado `padecimientos`:

	Paciente	LugarNacimiento	Edad	NivelEducativo	EstadoCivil	Alergias	Hipertensión	Diabetes
0	Juan Salgado	México	34	Preparatorio	Casado	1	1	0
1	Roberto Macias	Guanajuato	56	Secundaria	Casado	1	1	1
2	Laura Canavaro	Tamaulipas	76	Profesional	Casada	0	0	1
3	Hayde Cervantes	Guerrero	49	Profesional	Divorciada	1	0	0
4	Luis Vela	México	51	Secundaria	Casado	0	1	1
5	Monica Madrigal	Guadalajara	40	Maestría	Divorciada	1	0	0

Inicialmente, debes crear la función:

```
def etiqueta_persona(fila):  
    if fila['Edad'] > 60:  
        texto = 'Adulto '  
    else:  
        texto = 'Joven '  
  
    suma = fila['Alergias'] + fila['Hipertensión'] + fila['Diabetes']  
    if suma == 0:  
        return texto + 'sin padecimientos'  
    else:  
        return texto + 'con ' + str(suma) + ' padecimientos'
```

Observa que recibe una fila (registro) del *dataframe* e internamente se trabaja con la misma estructura de columnas que posee el *dataframe* original. La función se encarga de detectar si la persona es adulto o joven y después determina cuántas enfermedades.

Para crear la columna derivada, llamaremos a la función `apply()`, en donde, a su vez, se especifica la función `lambda` previamente definida. En la especificación se emplea la palabra reservada `lambda`, el nombre de la variable que se usará internamente para referirse a cada fila del *dataframe* (en este caso, `row`), dos puntos y, a continuación, la llamada a la función `etiqueta_persona()`, a la que se pasa como parámetro la misma variable que utilizaste para identificar las filas. Finalmente, como último parámetro de la función `apply()`, se indica la forma en que obtendrás los datos del *dataframe*, en este caso utilizamos `axis=1` para aplicar la función a cada fila. Entonces tendrías lo siguiente:

```
padecimientos['Texto Padecimiento'] = padecimientos.apply(lambda
row: etiqueta_persona(row), axis=1)
padecimientos
```

	Paciente	LugarNacimiento	Edad	NivelEducativo	EstadoCivil	Alergias	Hipertensión	Diabetes	Texto Padecimiento
0	Juan Salgado	México	34	Preparatorio	Casado	1	1	0	Joven con 2 padecimientos
1	Roberto Macias	Guanajuato	56	Secundaria	Casado	1	1	1	Joven con 3 padecimientos
2	Laura Canavaro	Tamaulipas	76	Profesional	Casada	0	0	1	Adulto con 1 padecimientos
3	Hayde Cervantes	Guerrero	49	Profesional	Divorciada	1	0	0	Joven con 1 padecimientos
4	Luis Vela	México	51	Secundaria	Casado	0	1	1	Joven con 2 padecimientos
5	Monica Madrigal	Guadalajara	40	Maestría	Divorciada	1	0	0	Joven con 1 padecimientos

Al final del *dataframe* aparece la nueva columna (`Texto Padecimiento`), con los valores que retorna la función `lambda` en dependencia de los datos de cada registro.

Resúmenes

Ahora que ya eres capaz de seleccionar adecuadamente la información y obtener columnas derivadas a partir de las originales, la siguiente tarea importante es generar resúmenes. Esta labor es fundamental en el proceso de ciencia de datos para extraer conocimiento de la información que se procesa o lograr un mejor entendimiento de tus datos.

Cuando procesas un *dataframe*, hay varias funciones disponibles que generan resultados alineados a tus necesidades. Se pueden ejecutar operaciones empleando una o varias columnas, o puedes centrarte en las filas y sus valores, ya que las funciones estadísticas aplican a ambos casos. Únicamente tienes que poner especial cuidado en que los cálculos se efectúen sobre valores operables numéricamente. A continuación, aprenderás a realizarlos de manera rápida y eficiente.

El proceso más común es trabajar con todos los datos de una columna. Por ejemplo, si deseas obtener un promedio, se aplica la función `mean()` a la columna deseada. Ahora, hagámoslo retomando la temática de *Star Wars* en donde se utiliza la estatura de los personajes:

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
print("La altura promedio de los personajes de star wars es: ",
personajesSW["height"].mean())
```

La altura promedio de los personajes de star wars es: 171.816091954023

Este tipo de funciones también es aplicable a varias columnas, de la siguiente forma:

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
personajesSW[["height", "mass"]].median()
```

```
height    178.0
mass       80.0
dtype: float64
```


Debes considerar que, al ser un cálculo en varias columnas, tendrás como resultado una estructura con la misma cantidad de columnas que operes, en este caso, la mediana de la altura y la mediana del peso de cada personaje.

Además, podrías aplicar varias funciones a una o más columnas a la vez. Para ello, puedes emplear el método `agg()` sobre el *dataframe*. Este recibe un diccionario, donde cada par de valores debe ser compuesto por el nombre de la columna y una lista con las funciones a efectuar en dicha columna.

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
personajesSW.agg(
    {
        "height": ["min", "max", "median", "skew"],
        "mass": ["min", "max", "median", "mean"],
    }
)
```

	height	mass
max	264.000000	1358.000000
mean	NaN	94.245977
median	178.000000	80.000000
min	45.000000	15.000000
skew	-1.218196	NaN

Estas funciones también pueden ser aplicadas a los resultados de un agrupamiento, es decir, al ejecutar alguna función sobre una columna en grupos determinados por el valor de otra columna. En otras palabras, los registros de una columna se dividen en grupos, según el contenido de la segunda. Por ejemplo, si deseas obtener el promedio de peso de los personajes de *Star Wars*, pero agrupados por su género, entonces tendrías lo siguiente:

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
personajesSW[["gender", "mass"]].groupby("gender").mean()
```

mass	
gender	
female	63.642105
hermaphrodite	1358.000000
male	85.374194
none	100.000000

Otra función muy provechosa es determinar el número de registros por categoría. Esto se realiza a través de `value_counts()`. Por ejemplo, si deseas saber cuáles son las categorías de género de los personajes de *Star Wars* y cuántos personajes caen en cada categoría, tendríamos:

```
import pandas as pd
personajesSW = pd.read_csv("/content/drive/MyDrive/Colab
Notebooks/M2DCData/characters.csv")
personajesSW["gender"].value_counts()
```

```
male          62
female        19
none           2
hermaphrodite  1
Name: gender, dtype: int64
```

Para conocer otras funciones estadísticas, te invitamos a consultar el material adicional de este documento.

Retomando nuestro planteamiento inicial donde tenemos, en un archivo de Excel, almacenados los pacientes de toda la República Mexicana con sus datos generales; y en diversas columnas, las enfermedades que pueden ser peligrosas al contraer alguna nueva enfermedad mortal, podríamos tener la necesidad de generar una nueva columna donde se indique si es urgente (que sea mayor de 60 años y tenga alguna de las enfermedades peligrosas) o no vacunar al paciente, ya que no es un paciente de riesgo y puede esperar. Si cargamos los datos en un *dataframe* llamado `pacientes`, podemos observar los datos:

	Nombre	Edad	Localización Geográfica	Diabetes	Obesidad	Hipertensión	Inmunosupresión
0	Juan Salgado	34	México	0	1	1	0
1	Roberto Macias	66	Guanajuato	1	1	1	0
2	Laura Canavaro	76	Tamaulipas	1	0	0	0
3	Hayde Cervantes	49	Guerrero	0	1	0	1
4	Luis Vela	51	México	1	0	1	1
5	Monica Madrigal	63	Guadalajara	0	0	0	0
6	Sandra Espinosa	54	Michoacan	0	1	1	0

Al tener una condición compleja que trabaja con los datos de varias columnas, lo mejor que puedes hacer es una función que se encargue de todas las comparaciones y de la determinación final del resultado. Así podrías emplearla como una función `lambda` en la creación de la nueva columna derivada.

```
def determinaVacuna(fila):  
    suma = fila['Diabetes'] + fila['Obesidad'] + fila['Hipertensión']  
    + fila['Inmunosupresión']  
    if fila['Edad'] > 60 and suma > 0:  
        return 'Si'  
    else:  
        return 'No'
```

```
pacientes['Vacunación Urgente'] = pacientes.apply(lambda row:  
determinaVacuna(row), axis=1)  
pacientes
```

	Nombre	Edad	Localización Geográfica	Diabetes	Obesidad	Hipertensión	Inmunosupresión	Vacunación Urgente
0	Juan Salgado	34	México	0	1	1	0	No
1	Roberto Macias	66	Guanajuato	1	1	1	0	Si
2	Laura Canavaro	76	Tamaulipas	1	0	0	0	Si
3	Hayde Cervantes	49	Guerrero	0	1	0	1	No
4	Luis Vela	51	México	1	0	1	1	No
5	Monica Madrigal	63	Guadalajara	0	0	0	0	No
6	Sandra Espinosa	54	Michoacan	0	1	1	0	No

Ahora que tienes una base para hacer cálculos con información numérica, es fundamental que conozcas la forma de trabajar con diferentes tipos de datos, como son los textos y las fechas. Esto lo veremos en el siguiente subtema.

| Manipulando texto y series de tiempo

Considera ahora un *dataframe* que contiene información de los medicamentos para combatir la influenza estacional. Entre los datos que se almacenan están: el nombre del medicamento, las sustancias activas, su presentación y las dosis recomendada. La compra de los medicamentos se hará a partir de dicha lista; sin embargo, la Secretaría de Salud ha prohibido la distribución de una de las sustancias activas para los pacientes con influenza estacional en el país.

¿Qué pasaría si los medicamentos tienen varias sustancias activas y todas ellas están registradas en una única columna? En estos casos, no podrías hacer una búsqueda o filtrado básico sobre la sustancia prohibida, por lo que necesitarás manipular campos de texto con mayor detalle.

En este sentido, **Pandas provee una gran variedad de funciones para el manejo de columnas de tipo texto. Con ellas podrás ejecutar cambios en los datos, hacer búsquedas u obtener información. Algunas de las más empleadas son: `lower()`, `upper()`, `title()`, `capitalize()`, `len()`, `contains()`, `cat()`, `replace()` y `split()`.** Aquí es muy importante mencionar que, en un *dataframe*, los textos se consideran como objetos, por lo que, para acceder a las funcionalidades de las cadenas, tendrás que utilizar `str` para que así se interprete de la forma: `nombre_dataframe.str.función_cadena`

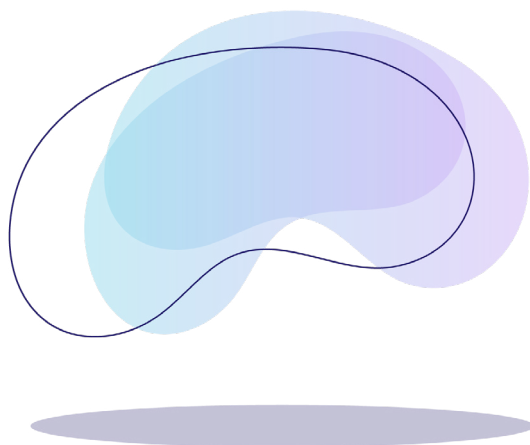
Para ejemplificar el empleo de las funciones de cadenas, utilizaremos el *dataframe* **species.csv** (localizado en el apartado “Archivos adjuntos” de la plataforma), el cual contiene información de las especies de los personajes de *Star Wars*, con 37 registros y 10 columnas:

	name	classification	designation	average_height	skin_colors	hair_colors	eye_colors	average_lifespan	language	homeworld
0	Hutt	gastropod	sentient	300.0	green, brown, tan	NaN	yellow, red	1000	Huttese	Nal Hutta
1	Yoda's species	mammal	sentient	66.0	green, yellow	brown, white	brown, green, yellow	900	Galactic basic	NaN
2	Trandoshan	reptile	sentient	200.0	brown, green	none	yellow, orange	NaN	Dosh	Trandosha
3	Mon Calamari	amphibian	sentient	160.0	red, blue, brown, magenta	none	yellow	NaN	Mon Calamarian	Mon Cala
4	Ewok	mammal	sentient	100.0	brown	white, brown, black	orange, brown	NaN	Ewokese	Endor
...	...	...	...	...	...	...	...	...	...	...
32	Pau'an	mammal	sentient	190.0	grey	none	black	700	Utapese	Utapau
33	Wookiee	mammal	sentient	210.0	gray	black, brown	blue, green, yellow, brown, golden, red	400	Shyriiwook	Kashyyyk
34	Droid	artificial	sentient	NaN	all	NaN	NaN	indefinite	NaN	NaN
35	Human	mammal	sentient	180.0	caucasian, black, asian, hispanic	blonde, brown, black, red	brown, blue, green, hazel, grey, amber	120	Galactic Basic	Coruscant
36	Rodian	sentient	reptilian	170.0	green, blue	NaN	black	NaN	Galactic Basic	Rodia

37 rows × 10 columns

Iniciemos con las funciones que permiten cambiar a mayúsculas o minúsculas todos los caracteres o algunos de ellos:

- `lower()`: Cambia todos los caracteres a minúsculas.
- `upper()`: Cambia todos los caracteres a mayúsculas.
- `title()`: Cambia a mayúsculas todos los inicios de las palabras.
- `capitalize()`: Cambia el primer caracter a mayúsculas y todos los demás a minúsculas.



Observa su comportamiento siendo aplicadas sobre la columna `language`:

```
especiesSW['language'].str.lower()
```

```
0          huttese
1    galactic basic
2          dosh
3    mon calamarian
4          ewokese
...
32         utapese
33    shyriiwook
34           NaN
35    galactic basic
36    galactic basic
```

```
especiesSW['language'].str.upper()
```

```
0          HUTTESE
1    GALACTIC BASIC
2          DOSH
3    MON CALAMARIAN
4          EWOKESE
...
32         UTAPESE
33    SHYRIIWOOK
34           NaN
35    GALACTIC BASIC
36    GALACTIC BASIC
```

```
especiesSW['language'].str.title()
```

```
0          Huttese
1    Galactic Basic
2          Dosh
3    Mon Calamarian
4          Ewokese
...
32         Utapese
33    Shyriiwook
34           NaN
35    Galactic Basic
36    Galactic Basic
```

```
especiesSW['language'].str.capitalize()
```

```
0          Huttese
1    Galactic basic
2          Dosh
3    Mon calamarian
4          Ewokese
...
32         Utapese
33    Shyriiwook
34           NaN
35    Galactic basic
36    Galactic basic
```

También pudieras tener la necesidad de conocer el tamaño de los textos que tiene alguna Serie. Esto puede resultarte útil cuando, por ejemplo, quieres guardar la información en una base de datos y necesitas especificar el tipo de campo a definir para dichos datos; o incluso cuando vas a presentar en pantalla la información y tienes espacios reducidos. Conocer los tamaños te ayudará a hacer los ajustes correspondientes. En cualquier caso, puedes utilizar la función `len()`, la cual te regresa la cantidad de caracteres que tiene una cadena de texto. En el siguiente ejemplo, está aplicada a la columna `name` y se ha creado una nueva columna derivada llamada `long`, con la longitud de cada nombre de especie:

```
especiesSW['long'] = especiesSW['name'].str.len()  
especiesSW[['name', 'long']]
```

	name	long
0	Hutt	4
1	Yoda's species	14
2	Trandoshan	10
3	Mon Calamari	12
4	Ewok	4
...	...	...
32	Pau'an	6
33	Wookiee	7
34	Droid	5
35	Human	5
36	Rodian	6

37 rows × 2 columns

Otra función útil es `contains()`, que te permite saber si una palabra o conjunto de caracteres se encuentra dentro de la cadena. Esta recibe como parámetro la palabra a buscar; regresa `True`, si la encuentra; `False`, en caso contrario. Es una función sensible a mayúsculas y minúsculas, aunque esta opción puede ser habilitada o deshabilitada utilizando como segundo parámetro el atributo `case`, que toma valores de `True` o `False`, respectivamente.

A continuación, aplicamos la búsqueda de la palabra 'blue' en la columna `skin_colors`:

```
especiesSW['skin_colors'].str.contains('blue', case=False)
```

```
0    False
1    False
2    False
3     True
4    False
...
32   False
33   False
34   False
35   False
36     True
```

```
Name: skin_colors, Length: 37, dtype: bool
```

Observa que solamente regresa una Serie con valores booleanos.

Como ya habrás identificado, puedes emplear esta información para filtrar los registros del *dataframe* como sigue:

```
especiesSW[especiesSW['skin_colors'].str.contains('Blue',
case=False)]
```

	name	classification	designation	average_height	skin_colors	hair_colors	eye_colors	average_lifespan	language	homeworld	long
3	Mon Calamari	amphibian	sentient	160.0	red, blue, brown, magenta	none	yellow	NaN	Mon Calamarian	Mon Cala	12
8	Toydarian	mammal	sentient	120.0	blue, green, grey	none	yellow	91	Toydarian	Toydaria	9
10	Twilek	mammals	sentient	200.0	orange, yellow, blue, green, pink, purple, tan	none	blue, brown, orange, pink	NaN	Twileki	Ryloth	7
11	Aleena	reptile	sentient	80.0	blue, gray	none	NaN	79	Aleena	Aleen Minor	6
16	Nautolan	amphibian	sentient	180.0	green, blue, brown, red	none	black	70	Nautila	Glee Anselm	8
22	Chagrian	amphibian	sentient	190.0	blue	none	blue	NaN	Chagria	Champala	8
27	Kaminoan	amphibian	sentient	220.0	grey, blue	none	black	80	Kaminoan	Kamino	8
30	Togruta	mammal	sentient	180.0	red, white, orange, yellow, green, blue	none	red, orange, yellow, green, blue, black	94	Togruti	Shili	7
36	Rodian	sentient	reptilian	170.0	green, blue	NaN	black	NaN	Galactic Basic	Rodia	6

Por su parte, la función `cat()` se utiliza para **concatenar** cadenas de textos, así podrás combinar el contenido de algunas columnas. Por ejemplo, puedes generar una Serie con el nombre de la especie (`name`) seguido de la frase ' is a ', más los valores de la clasificación (`classification`).

```
especiesSW['name'].str.cat(' is a ' + especiesSW['classification'])
```

```
0          Hutt is a gastropod
1    Yoda's species is a mammal
2    Trandoshan is a reptile
3    Mon Calamari is a amphibian
4          Ewok is a mammal
...
32          Pau'an is a mammal
33          Wookiee is a mammal
34          Droid is a artificial
35          Human is a mammal
36          Rodian is a sentient
Name: name, Length: 37, dtype: object
```

Otra forma en la que puedes emplear `cat()` es para generar una sola cadena a partir de todos los valores de una Serie, simplemente aplicándola a la columna en cuestión. Probemos nuevamente con `name` y como parámetro indicamos el separador que se utilizará entre cada elemento de la Serie, a través del atributo `sep`. En este caso, utilizaremos comas (,) para obtener una cadena con todos los nombres de las especies:

```
especiesSW['name'].str.cat(sep=", ")
```

Hutt, Yoda's species, Trandoshan, Mon Calamari, Ewok, Sullustan, Neimodian, Gungan, Toydarian, Dug, Twi'lek, Aleena, Vulptereen, Xexto, Toong, Cerean, Nautolan, Zabrak, Tholothian, Iktotchi, Quermian, Kel Dor, Chagrian, Geonosian, Mirialan, Clawdite, Besalisk, Kaminoan, Skakoan, Muun, Togruta, Kaleesh, Pau'an, Wookiee, Droid, Human, Rodian

Para realizar **reemplazos** de algunos o todos los datos, puedes emplear `replace()`. La primera forma en la que puedes utilizar esta función es aplicarla a una Serie y mandando como parámetro un diccionario donde cada par `llave:valor` indique **"cadena a buscar": "cadena de reemplazo"**. Por ejemplo, si quieres cambiar los valores de la columna `designation`, la cual posee únicamente los valores `sentient` y `reptilian`, por `S` o `R`, según sea el caso, podrías hacer:

```
especiesSW['short_designation'] = especiesSW['designation'].  
replace({"sentient": "S", "reptilian": "R"})  
especiesSW[['designation', 'short_designation']]
```

	designation	short_designation
0	sentient	S
1	sentient	S
2	sentient	S
3	sentient	S
4	sentient	S
...	...	...
32	sentient	S
33	sentient	S
34	sentient	S
35	sentient	S
36	reptilian	R

37 rows × 2 columns

Observa que no se empleó `str`; es decir, en este uso, `replace()` está definida sobre las Series.

Otra forma de emplear `replace()` es a través de una función en la que programes exactamente qué pasará con la cadena. En esta opción, `replace()` puede aceptar una expresión regular en lugar de una palabra a buscar.

En el siguiente fragmento de *script*, se ha creado una expresión regular para buscar todas las cadenas que comienzan con T (`^T.*`). Para aquellos registros que cumplan con el patrón, se mandará la cadena a la función `procesaTexto`. Se especifica el atributo `regex=True` para indicar que es una verificación a través de una expresión regular.

A manera de ejemplo, en la función `procesaTexto` únicamente se han cambiado los caracteres a mayúsculas, pero podrías realizar cualquier proceso. Además, para obtener el texto, se utiliza la función `group()`, ya que lo que llega a la función es un objeto de tipo `re.Match`, que son los resultados de la verificación de las expresiones regulares. Como resultado, solamente los nombres de los mundos de la columna `homeworld` que comienzan con T, se cambian a mayúsculas.

```
def procesaTexto(texto):  
    return texto.group().upper()  
  
expReg = "(^T.*)"  
especiesSW['homeworld change'] = especiesSW['homeworld'].  
str.replace(expReg, procesaTexto, regex=True)  
especiesSW['homeworld change']
```

```
0      NaI Hutta  
1          NaN  
2      TRANDOSHA  
3      Mon Cala  
4          Endor  
      ...  
32      Utapau  
33      Kashyyyk  
34          NaN  
35      Coruscant  
36      Rodia  
Name: homeworld change, Length: 37, dtype: object
```


Si deseas profundizar más sobre las expresiones regulares, te recomendamos revisar el material adicional de este documento.

Otro método comúnmente utilizado es `split()`, que te permite **separar** una cadena de texto en cadenas más pequeñas delimitadas por un separador. Las cadenas obtenidas formarán parte de una lista que te permitirá manejarlas de manera individual. El separador puede estar conformado por uno o varios caracteres. Por ejemplo, si observas la columna `skin_colors`, te percatarás de que posee varios colores posibles para la especie. Si deseas manipular cada uno de manera individual, puedes aplicar un `split()` para que cada color sea incluido como un elemento de la lista, utilizando el separador `,`, ya que es la forma en que separaron la lista de manera original.

```
especiesSW['skin_colors_list'] = especiesSW['skin_colors'].str.  
split(',')  
especiesSW['skin_colors_list']
```

```
0          [green, brown, tan]  
1          [green, yellow]  
2          [brown, green]  
3  [red, blue, brown, magenta]  
4          [brown]  
  
...  
32          [grey]  
33          [gray]  
34          [all]  
35  [caucasian, black, asian, hispanic]  
36          [green, blue]  
Name: skin_colors_list, Length: 37, dtype: object
```

Si deseas conocer el tamaño de la lista, puedes utilizar nuevamente `str` para llamar a la función `len()`.

```
especiesSW['skin_colors_list_len'] = especiesSW['skin_colors'].  
str.split(', ').str.len()  
especiesSW[['skin_colors_list', 'skin_colors_list_len']]
```

	skin_colors_list	skin_colors_list_len
0	[green, brown, tan]	3
1	[green, yellow]	2
2	[brown, green]	2
3	[red, blue, brown, magenta]	4
4	[brown]	1
...	...	...
32	[grey]	1
33	[gray]	1
34	[all]	1
35	[caucasian, black, asian, hispanic]	4
36	[green, blue]	2

37 rows × 2 columns

Te recomendamos consultar la documentación del material adicional de este documento, donde encontrarás todas las funciones de cadenas, empleando la siguiente ruta: **pandas > Series > str > función_a_consultar**.

Esta documentación te lleva a la primera función (`capitalize()`), por lo que puedes revisar el resto avanzando con el botón **“Next” (>)** o navegando en el menú de la izquierda para seleccionar la función de tu interés.

Además de las columnas numéricas y textuales, el manejo de **fechas y tiempos** es muy importante cuando estás manipulando los datos en un *dataframe*. Pandas cuenta con funciones avanzadas para el manejo de este tipo de datos, y se basa en los tipos de NumPy. Dos de los más utilizados son:

1. Fechas y horas específicas con soporte para zona horaria con el tipo `datetime64[ns]`
2. Una duración de tiempo absoluta con el tipo `timedelta64[ns]`



Cuando vas a trabajar con fechas y horas, y extraes la información de un archivo, puedes indicarle a la función de lectura que hay campos que deseas sean considerados como tal, a través del atributo `parse_dates`, y asignándole una lista con los nombres de dichos campos. Por ejemplo, si leemos un archivo de Excel denominado **Sales.xlsx** (descárgalo desde el apartado “Archivos adjuntos” de la plataforma), y tiene dos columnas con fechas llamadas: `Order ID` y `Ship Date`, entonces podrías realizar lo siguiente:

```
ventas = pd.read_excel("/content/drive/MyDrive/Colab
Notebooks/M2DCData/Sales.xlsx", parse_dates=["Order ID",
"Ship Date"])
ventas
```

	Row ID	Order ID	Order Date	Order Priority	Order Quantity	...	Product Sub-Category	Product Name	Product Container	Product Base Margin	Ship Date	
	0	1	3	2010-10-13	Low	6	...	Storage & Organization	Eldon Base for stackable storage shelf, platinum	Large Box	0.80	2010-10-20
	1	49	293	2012-10-01	High	49	...	Appliances	1.7 Cubic Foot Compact "Cube" Office Refrigera...	Jumbo Drum	0.58	2012-10-02
	2	50	293	2012-10-01	High	27	...	Binders and Binder Accessories	Cardinal Slant-D® Ring Binder, Heavy Gauge Vinyl	Small Box	0.39	2012-10-03
	3	80	483	2011-07-10	High	30	...	Telephones and Communication	R380	Small Box	0.58	2011-07-12
	4	85	515	2010-08-28	Not Specified	19	...	Appliances	Holmes HEPA Air Purifier	Medium Box	0.50	2010-08-30
	...	...	...	...	...	...	...	...	...	...	...	...
	8394	7765	55558	2010-08-09	Medium	8	...	Bookcases	Bush Mission Pointe Library	Jumbo Box	0.65	2010-08-09
	8395	7766	55558	2010-08-09	Medium	23	...	Envelopes	Recycled Interoffice Envelopes with Re-Use-A-S...	Small Box	0.38	2010-08-11
	8396	7906	56550	2011-04-08	Not Specified	37	...	Office Furnishings	Executive Impressions 14"	Small Pack	0.41	2011-04-10
	8397	7907	56550	2011-04-08	Not Specified	8	...	Telephones and Communication	Talkabout T8367	Small Box	0.56	2011-04-09
	8398	7914	56581	2009-02-08	High	20	...	Office Furnishings	Tenex 46" x 60" Computer Anti-Static Chairmat,...	Medium Box	0.65	2009-02-11

En la información del *dataframe*, las columnas `Order ID` y `Ship Date` quedan con el tipo `datetime64[ns]`, lo cual te permitirá manipular las fechas de una manera muy sencilla y te proporcionará opciones para realizar operaciones con dichos campos.

```
ventas.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 8399 entries, 0 to 8398
```

```
Data columns (total 21 columns):
```

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	Row ID	8399 non-null	int64
1	Order ID	8399 non-null	int64
2	Order Date	8399 non-null	datetime64[ns]
3	Order Priority	8399 non-null	object
4	Order Quantity	8399 non-null	int64
5	Sales	8399 non-null	float64
6	Discount	8399 non-null	float64
7	Ship Mode	8399 non-null	object
8	Profit	8399 non-null	float64
9	Unit Price	8399 non-null	float64
10	Shipping Cost	8399 non-null	float64
11	Customer Name	8399 non-null	object
12	Province	8399 non-null	object
13	Region	8399 non-null	object
14	Customer Segment	8399 non-null	object
15	Product Category	8399 non-null	object
16	Product Sub-Category	8399 non-null	object
17	Product Name	8399 non-null	object
18	Product Container	8399 non-null	object
19	Product Base Margin	8336 non-null	float64
20	Ship Date	8399 non-null	datetime64[ns]

```
dtypes: datetime64[ns] (2), float64(6), int64(3), object(10)
```

```
memory usage: 1.3+ MB
```

En muchas ocasiones, Pandas puede hacer la detección automática del tipo, aun si no se ha especificado explícitamente en la función de lectura. Pero si esto no sucede, y al revisar los tipos de datos no son `datetime64[ns]`, entonces puedes **convertir las columnas a tipo fecha** por medio de la función `to_datetime()`. Para el ejemplo anterior, si las columnas no hubieran sido leídas como fecha, podrías realizar lo siguiente para convertirlas:

```
ventas['Order Date'] = pd.to_datetime(ventas['Order Date'])
ventas['Ship Date'] = pd.to_datetime(ventas['Ship Date'])
```

Ahora puedes manipular fácilmente las columnas de tipo fecha y aplicarles operadores y funciones. Por ejemplo, puedes aplicar las funciones `min()` y `max()` a la columna `Order Date` para conocer las fechas de la primera y última orden, respectivamente. También puedes restar la primera fecha a la última para saber la cantidad de tiempo en la que se han realizado todas las órdenes.

```
print('Día del primer pedido: ', ventas['Order Date'].min())
print('Día del último pedido: ', ventas['Order Date'].max())
ventas['Order Date'].max() - ventas['Order Date'].min()
```

```
Día del primer pedido: 2009-01-01 00:00:00
Día del último pedido: 2012-12-30 00:00:00
Timedelta('1459 days 00:00:00')
```

Observa que la diferencia entre las fechas está expresada con el tipo de dato `TimeDelta`, que sirve para expresar diferencias de tiempo.

Al tener las fechas de orden y de despacho en el *dataframe*, puedes obtener la cantidad promedio de tiempo en que se atienden las órdenes. Para ello, iniciemos creando una columna derivada denominada `dispatch`, que contenga la diferencia de tiempo entre la fecha de envío y la fecha de orden. Este dato se almacenará con un tipo `Timedelta`. Finalmente, aplica la función promedio `mean()` a dicha columna:

```
ventas['dispatch'] = ventas['Ship Date'] - ventas['Order Date']
ventas['dispatch'].mean()
```

```
Timedelta('2 days 00:47:50.055959042')
```

Con las columnas como fechas, también puedes extraer el día, mes y año utilizando los atributos `day`, `month` y `year`, respectivamente, del objeto `dt` (`datetime`). Observa que los datos obtenidos se manejan como enteros:

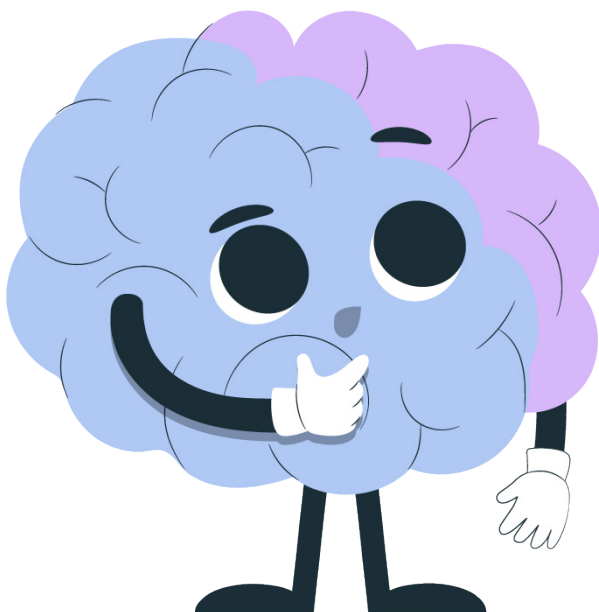
<code>ventas['Order Date'].dt.day</code>		<code>ventas['Order Date'].dt.month</code>		<code>ventas['Order Date'].dt.year</code>	
0	13	0	10	0	2010
1	1	1	10	1	2012
2	1	2	10	2	2012
3	10	3	7	3	2011
4	28	4	8	4	2010
	..		..		...
8394	9	8394	8	8394	2010
8395	9	8395	8	8395	2010
8396	8	8396	4	8396	2011
8397	8	8397	4	8397	2011
8398	8	8398	2	8398	2009

Con la separación anterior, puedes agrupar todas las órdenes por año y mes, por ejemplo, para obtener la cantidad de órdenes que se realizan por día:

```
ventas['Order Date'].groupby([ventas['Order Date'].dt.year,
ventas['Order Date'].dt.month]).value_counts()
```

Order Date	Order Date	Order Date	
2009	1	2009-01-05	15
		2009-01-21	14
		2009-01-06	13
		2009-01-10	12
		2009-01-14	12
		..	
2012	12	2012-12-23	2
		2012-12-26	2
		2012-12-14	1
		2012-12-17	1
		2012-12-22	1

Name: Order Date, Length: 1418, dtype: int64



Puedes consultar información adicional para el manejo de fechas y tiempos en el material adicional de este documento.

Retomando el planteamiento inicial, con el *dataframe* que contiene los nombres de los medicamentos para combatir la influenza estacional, en donde se utiliza el nombre, las sustancias activas, la presentación y la dosis, se busca identificar qué medicamentos contiene la sustancia activa hidroxiclороquina, prohibida en México por la Secretaría de Salud. Un fragmento del *dataframe*, denominado `medicamentos2`, se muestra a continuación:

	Medicamento	Substancias Activas	Presentación	Dosis
0	Paracetamol	Paracetamol	Tabletas	1 tableta cada 8 hrs
1	Dolquine	Sulfato de hidroxiclороquina	Comprimidos	2 a 3 comprimidos al día
2	Naproxeno	Paracetamol	Tabletas	1 tableta cada 8 hrs
3	Actron	Ibuprofeno	Comprimidos	1 tableta cada 8 hrs
4	Plaquenil plus	Hidroxiclороquina y paracetamol	Tabletas	2 a 3 comprimidos al día
5	Quensyl	Hidroxiclороquina	Tabletas	1 tableta en la comida
6	ClamoxinS	Amoxicilina	Suspensión	1 cucharada cada 12 hrs

Para encontrar los medicamentos que contengan hidroxiclороquina en cualquiera de sus presentaciones, puedes utilizar la función `contains()` con el parámetro `case` en `False` (para que busque la palabra sin importar las mayúsculas y minúsculas) y utilizar el resultado como condición de indexación para obtener los registros completos:

```
medicamentos2[medicamentos2['Substancias Activas'].str.contains('hidroxiclороquina', case=False)]
```

	Medicamento	Substancias Activas	Presentación	Dosis
1	Dolquine	Sulfato de hidroxiclороquina	Comprimidos	2 a 3 comprimidos al día
4	Plaquenil plus	Hidroxiclороquina y paracetamol	Tabletas	2 a 3 comprimidos al día
5	Quensyl	Hidroxiclороquina	Tabletas	1 tableta en la comida

Ahora con estos métodos que trabajan con diferentes tipos de datos, puedes manipular y procesar la información de tu *dataframe*, pero, con el surgimiento y la fuerza que ha tomado la Inteligencia Artificial, es importante conocer cómo puede utilizarse con la plataforma Pandas, lo que podrás conocer más adelante.

| Antes de terminar...

Al iniciar esta lección, ya seleccionabas información de tus *dataframes*, permitiéndote reducir sus dimensiones. Ahora realizas cálculos y creas resúmenes para tener una visión generalizada de tus datos, teniendo como posibilidad almacenar los resultados en el propio *dataframe* o en uno nuevo, dependiendo de tus requerimientos o cuando los recursos o tiempos de respuesta sean una limitante.

Recuerda que la revolución de la inteligencia artificial ha alcanzado las herramientas de desarrollo de aplicaciones para ciencia de datos y es muy fácil implementarla en tus desarrollos.

Ahora reflexiona si eres capaz de:

- Crear columnas derivadas a partir de la información de otras columnas para extender el contenido del *dataframe*.
- Manipular la información del *dataframe* para crear resúmenes significativos, ejecutando cálculos sobre los datos.

No olvides siempre consultar la documentación oficial cuando requieras operar cadenas de texto y/o series de tiempo. Pandas ofrece una gran variedad de funciones para ello.

| Material adicional

Expresiones regulares

Python Software Foundation. (2025). *re — Regular expression operations* [re— Operaciones con expresiones regulares]. Python. Recuperado el 26 de mayo de 2025 de: <https://docs.python.org/3/library/re.html>

Funciones de cadenas

Pandas vía. (2024). *pandas.Series.str.capitalize*. Pandas. Recuperado el 26 de mayo de 2025 de: <https://pandas.pydata.org/docs/reference/api/pandas.Series.str.capitalize.html>

Funciones estadísticas

Pandas vía. (2024). *Descriptive statistics* [Estadísticas descriptivas]. Pandas. Recuperado el 26 de mayo de 2025 de: https://pandas.pydata.org/docs/user_guide/basics.html#basics-stats

Manejo de fechas y tiempos

Pandas vía. (2024). *Time series / date functionality* [Funcionalidad de series de tiempo/fecha]. Pandas. Recuperado el 26 de mayo de 2025 de: https://pandas.pydata.org/docs/user_guide/timeseries.html?highlight=datetime

Nota: Los enlaces a las páginas web incluidos en este apartado fueron consultados y verificados por última vez el 26 de mayo de 2025 y su disponibilidad depende del autor.

Bibliografía

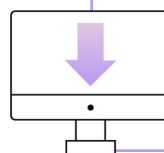
Pandas via. (2021). *Descriptive statistics* [Estadística descriptiva]. Pandas. Recuperado el 26 de mayo de 2025 de: https://pandas.pydata.org/docs/user_guide/basics.html#basics-stats

Pandas via. (20 de septiembre de 2024). *Pandas documentation* [Documentación de pandas]. Pandas. Recuperado el 26 de mayo de 2025 de: <https://pandas.pydata.org/docs/index.html>

Pandas vía. (2024). *pandas.Series.str.capitalize*. Pandas. Recuperado el 26 de mayo de 2025 de: <https://pandas.pydata.org/docs/reference/api/pandas.Series.str.capitalize.html>

Pandas vía. (2024). *Time series / date functionality* [Funcionalidad de series de tiempo/fecha]. Pandas. Recuperado el 26 de mayo de 2025 de: https://pandas.pydata.org/docs/user_guide/timeseries.html?highlight=datetime

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 26 de mayo de 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.